

shapes being drawn would be too small to be seen on the screen and so the function should simply return.

We keep track of the turtle's original position and heading in three variables: `initialX`, `initialY`, and `initialHeading`. This way, no matter where the shape-drawing function leaves the turtle positioned or what direction it is headed, `drawFractal()` can revert the turtle back to the original position and heading for the next recursive call:

Python

```
# Save the position and heading at the start of this function call:
initialX = turtle.xcor()
initialY = turtle.ycor()
initialHeading = turtle.heading()
```

The `turtle.xcor()` and `turtle.ycor()` functions return the absolute x- and y-coordinates of the turtle on the screen. The `turtle.heading()` function returns the direction in which the turtle is pointed in degrees.

The next few lines call the shape-drawing function passed to the `shapeDrawFunction` parameter:

Python

```
# Call the draw function to draw the shape:
turtle.pendown()
shapeDrawFunction(size, depth)
turtle.penup()
```

Because the value passed as the argument for the `shapeDrawFunction` parameter is a function, the code `shapeDrawFunction(size, depth)` calls this function with the values in `size` and `depth`. The pen is lowered before and raised after the `shapeDrawFunction()` call to ensure that the shape-drawing function can consistently expect the pen to be down when the drawing begins.

Using the Specifications Dictionary

After the call to `shapeDrawFunction()`, the rest of `drawFractal()`'s code is devoted to making recursive calls to `drawFractal()` based on the specification in the